

Declaração de Uso de IA Generativa

Ferramenta e modelo utilizados

Claude Code (Anthropic), modelo Claude Sonnet 5, utilizado via CLI/extensão integrada ao editor.

Finalidade de cada uso

1. Apoiar a estruturação do repositório (organização de pastas, `docker-compose.yml`, scripts de automação de experimentos e coleta de métricas).
2. Redigir scripts de observação de processos/threads (`ps`, `pstree`, `docker stats`) e de chamadas de sistema (`strace`) a serem executados no container Linux do Ollama.
3. Redigir um script Python para automatizar as 12+ execuções comparativas das Configurações 1, 2 e 3 e gravar métricas em CSV.
4. Auxiliar na redação da estrutura do README, do relatório técnico e desta própria declaração.
5. Explicar conceitos de Sistemas Operacionais (escalonamento, threads, chamadas de sistema) usados na interpretação dos resultados.

Prompts relevantes (até 5)

1. "O `strace` dentro do container do Ollama está falhando com `ptrace(PTRACE_SEIZE, 1): Operation not permitted` — por que isso acontece em containers Docker e como resolvo sem rodar o container como privilegiado?"
2. "Quero comparar 1 requisição sequencial contra 4 requisições simultâneas usando a API do Ollama, e depois `num_ctx` 2048 contra 8192. Como estruturar um script Python que dispare essas requisições, use threads para a concorrência e grave tempo total, TTFT e uso de CPU/memória do container em CSV?"
3. "Nos logs do `llama-server` aparece muito `futex` no resumo do `strace -c`. Isso é normal para um processo de inferência com várias threads, ou indica algum problema de contenção?"
4. "O modelo oficial `meta-llama/Llama-3.2-3B-Instruct` no Hugging Face é gated e distribui só `safetensors`. Para rodar via Ollama localmente, faz sentido usar o build GGUF `llama3.2:3b` da biblioteca do próprio Ollama e considerar o mesmo modelo-base, ou isso muda a validade da minha reserva no Classroom?"
5. "Nos meus experimentos de concorrência, 6 das 8 requisições com concorrência=4 deram timeout em 300s — isso é esperado num notebook com 4 CPUs e sem GPU, ou é sinal de erro na forma como configurei o teste?"

Sugestões aproveitadas, corrigidas ou rejeitadas

- **Aproveitado:** uso de containers Docker (backend WSL2 do Docker Desktop) como "ambiente Linux", em vez de instalar uma distribuição WSL completa — mais rápido de configurar sem perda de validade, já que a Trilha A prevê explicitamente análise de "execução nativa ou em contêiner".
- **Corrigido/ajustado:** o *model card* de referência no Hugging Face (`meta-llama/Llama-3.2-3B-Instruct`) é um repositório com acesso restrito (*gated*) em formato *safetensors*; para a execução local, foi utilizado o build GGUF Q4_K_M distribuído pela biblioteca oficial do Ollama (`llama3.2:3b`), correspondente ao mesmo modelo-base. Essa diferença foi documentada explicitamente no relatório para preservar a rastreabilidade.
- **Verificado manualmente, não apenas aceito:** todos os comandos de coleta (`ps` , `docker stats` , `strace` , chamadas à API do Ollama) foram efetivamente executados no ambiente da discente, e os números apresentados no relatório vêm dos arquivos `data/*.txt` e `data/experimentos/*.csv` gerados por essas execuções reais — não foram inventados pela IA.

Erros encontrados

- O script de `strace` sugerido inicialmente pela IA falhou com `ptrace(PTRACE_SEIZE, 1): Operation not permitted` , porque containers Docker não concedem `ptrace` por padrão. Corrigido adicionando `cap_add: [SYS_PTRACE]` ao serviço `ollama` no `docker-compose.yml` .
- Depois de corrigir a permissão, o `strace` ainda falhava ao copiar o arquivo de resumo (`/tmp/strace-resumo.txt`) porque o Git Bash/MSYS no Windows converte automaticamente caminhos no estilo `/tmp/...` passados como argumento de linha de comando para um caminho do sistema de arquivos Windows. Corrigido definindo `MSYS_NO_PATHCONV=1` no script.
- Nos experimentos de concorrência (Configuração 2, concorrência=4), 6 das 8 requisições disparadas em paralelo estouraram o timeout de 300s definido no script Python — não foi um erro do script, mas um resultado real e esperado dado que o host tem apenas 4 CPUs e nenhuma GPU; esse comportamento foi mantido e discutido no relatório em vez de ser "escondido" aumentando artificialmente o timeout.

Testes, documentação ou observações usados para verificar respostas

- Execução direta dos comandos sugeridos no ambiente da discente (não apenas leitura).
- Conferência dos campos retornados pela API do Ollama (`total_duration` , `load_duration` , `eval_count` , `eval_duration`) com a [documentação oficial da API do Ollama](#).
- Conferência do model card oficial no Hugging Face para número de parâmetros e licença do modelo.
- Inspeção manual dos arquivos de log e CSV gerados pelos scripts antes de citá-los no relatório.